

USART

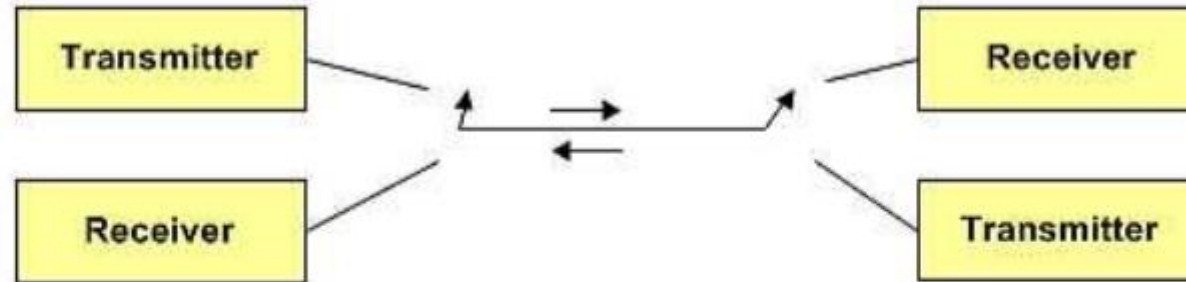
introdução

- Comunicação serial x paralela
- Comunicação serial: transmite um bit de cada vez.
 - Problema a ser resolvido: **sincronismo** - como o receptor sabe que chegou um novo bit?
 - **Serial síncrona**: transmissor envia um sinal (clock) que vai indicar a transmissão de um novo bit
 - **Serial assíncrona**: transmissor sinaliza que vai iniciar a transmissão de uma sequência de bits a intervalos constantes.
- Transmissão Half duplex x Full duplex

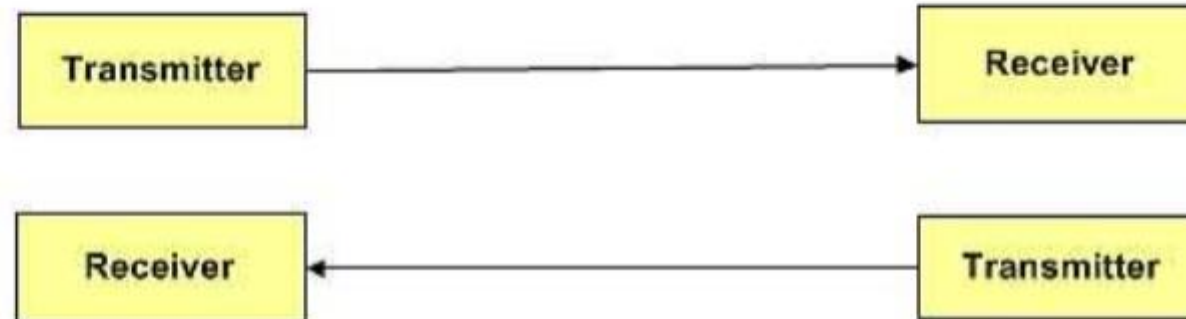
Simplex



Half Duplex



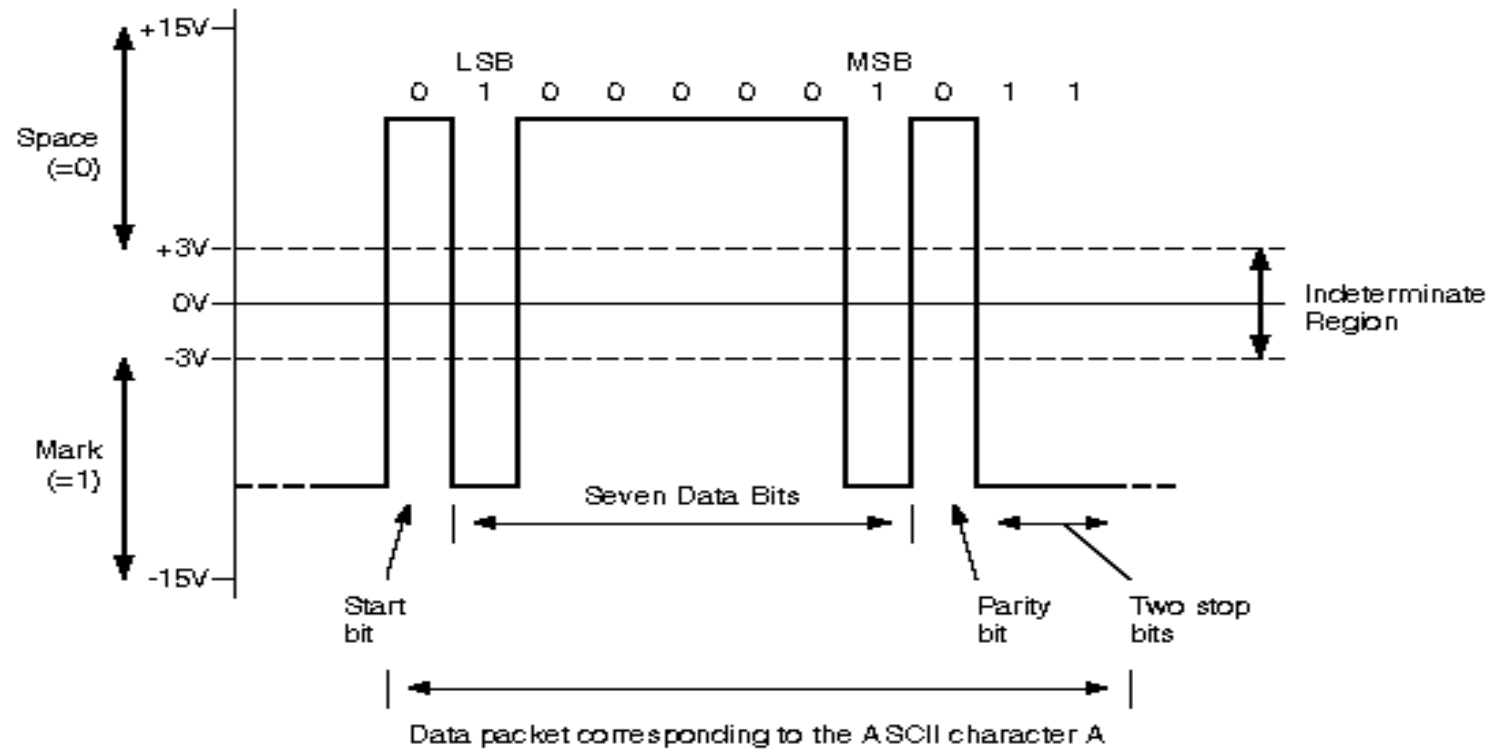
Full Duplex



EIA-232 – CARACTERÍSTICAS ELÉTRICAS

- A tensão de um circuito em aberto nunca deverá exceder 25 Volts (com referência ao terra (GND));
- Uma corrente de curto-circuito nunca deverá exceder 500mA. Os drivers devem ser capazes de manusear esta corrente sem danos.
- Outras características estão descritas no documento do padrão:
 - Ex: capacitância da linha, taxa máxima de transmissão, etc.

EIA-232 – CARACTERÍSTICAS ELÉTRICAS



EIA-232 – Opções de configurações

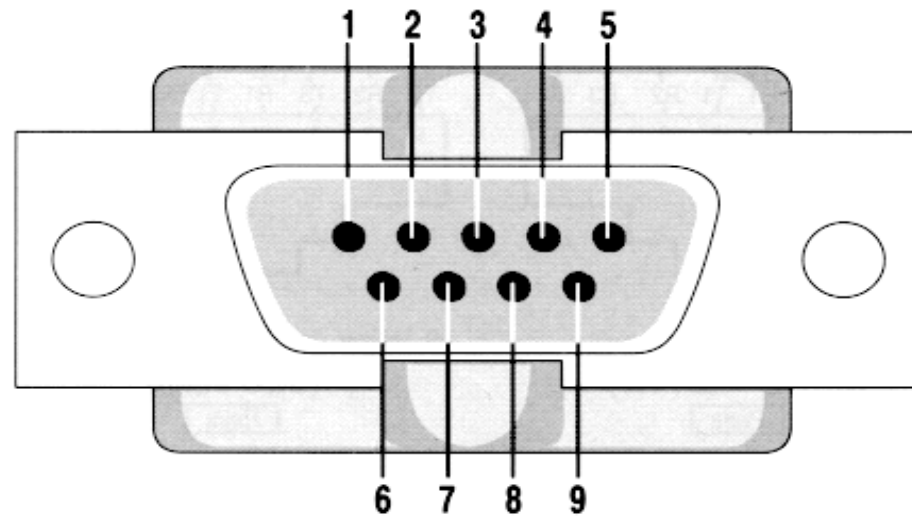
- **Paridade:**

- Paridade par: a soma dos bits de dados iguais a 1 mais o bit de paridade tem que ser par
- Paridade ímpar: a soma dos bits de dados iguais a 1 mais o bit de paridade ter que ser ímpar.
- Sem paridade: este bit não é enviado

EIA-232 – Opções de Configurações

- **Número de bits por carácter:** total de bits de dados transmitidos por vez (de 4 a 8 bits).
- **Número de StopBits:** tempo relativo a duração de um bit inserido ao final da transmissão. Necessário para criar um intervalo de tempo entre transmissões consecutivas. Pode ter a duração de 1 ou 2 bits.
- **Taxa de transferência ou baud rate:** é o total de bits que pode ser transferido por segundo (bps) e define a duração da transmissão de 1 bit = $1/\text{BaudRate}$. Existem alguns valores padrão que são usados na maioria das aplicações: 110, 300, 1200, 2400, 4800, 9600, 19.200, 38.400, 57.600 e 115.200.

RS-232 – CONECTORES – DB 9



Pin	Signal	Pin	Signal
1	Data Carrier Detect	6	Data Set Ready
2	Received Data	7	Request to Send
3	Transmitted Data	8	Clear to Send
4	Data Terminal Ready	9	Ring Indicator
5	Signal Ground		

RS-232 – CONTROLE DE FLUXO

- **O controle de fluxo, ou seja, o controle da transferência de dados podem ser de software ou de hardware;**
- **Software: normalmente denominado XON/XOFF, faz uso de dois caracteres (ASCII 17 e ASCII 19). Quando o modem estiver com o buffer cheio, ele envia um caracter de XOFF para o computador parar de enviar caracteres.**
- **Hardware: também conhecido como RTS/CTS, duas linhas de controle disponíveis na interface RS-232. Quando o computador deseja enviar dados, ele ativa a linha RTS. Se o modem pode receber dados, ele responde com um CTS e então o computador inicia o envio de dados.**

Eia-232 - usos

Apesar de originalmente ter sido criada para a comunicação entre o computador e um dispositivo de comunicação (Modem), a comunicação serial assíncrona está presente como forma de comunicação entre módulos que podem fazer parte do projeto embarcado que está sendo criado. Modems GPRS, módulos GPS, módulos Bluetooth são alguns exemplos de módulos que comumente se comunicam usando a serial assíncrona.

O que é importante ressaltar nestes casos é o não uso dos níveis de tensão originais do padrão EIA-232, sendo as transmissões do Mark e Space como níveis lógicos compatíveis com uma porta digital.

Padrão EIA-232 – parâmetros de configuração

- número de bits por *framing*
- número de stop bits
- bit de paridade
- taxa de transmissão (Baudrate)

Example 10-1

Calculate the total number of bits used in transferring 50 pages of text, each with 80×25 characters. Assume 8 bits per character and 1 stop bit.

Solution:

For each character a total of 10 bits is used, 8 bits for the character, 1 stop bit, and 1 start bit. Therefore, the total number of bits is $80 \times 25 \times 10 = 20,000$ bits per page. For 50 pages, 1,000,000 bits will be transferred.

Example 10-2

Calculate the minimal time it takes to transfer the entire 50 pages of data in Example 10-1 using a baud rate of:

- (a) 9600 (b) 57,600

Solution:

(a) $1,000,000 / 9600 = 104$ seconds

(b) $1,000,000 / 57,600 = 17$ seconds

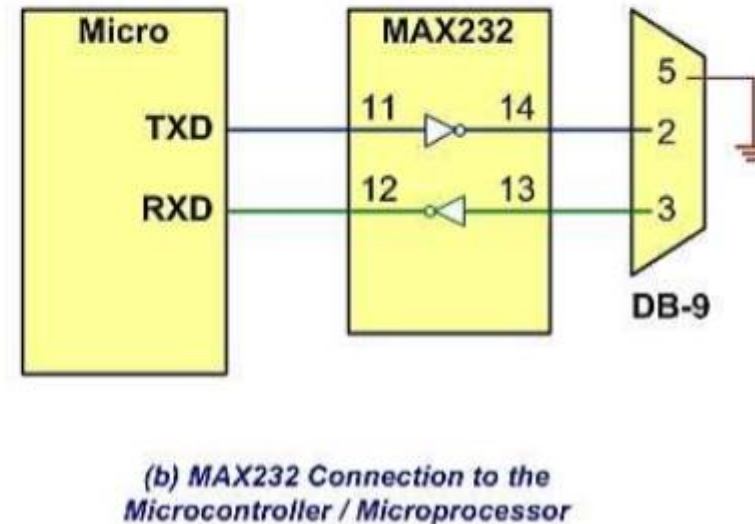
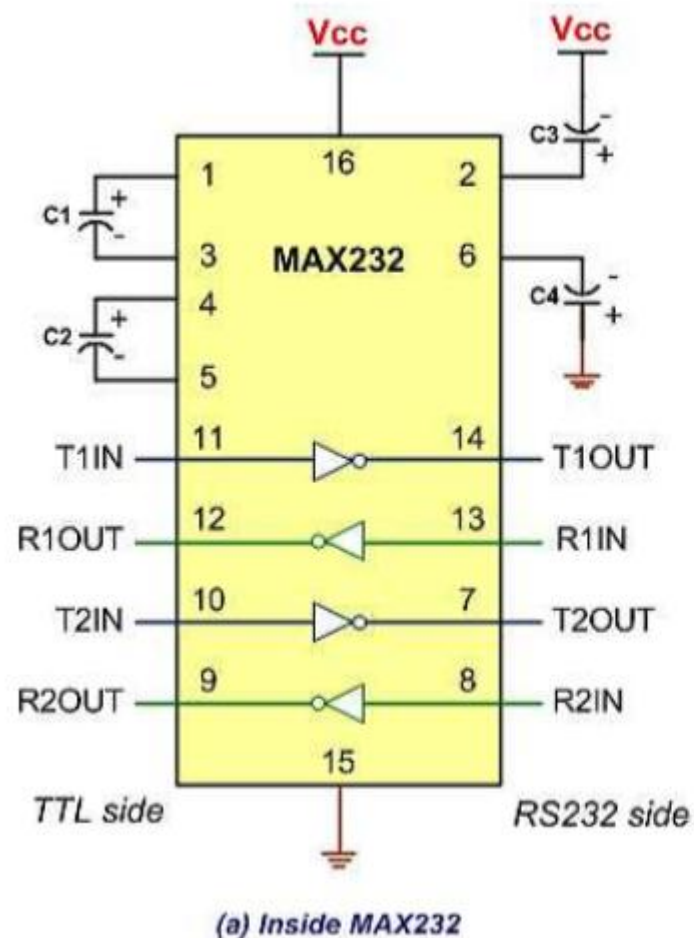
Example 10-3

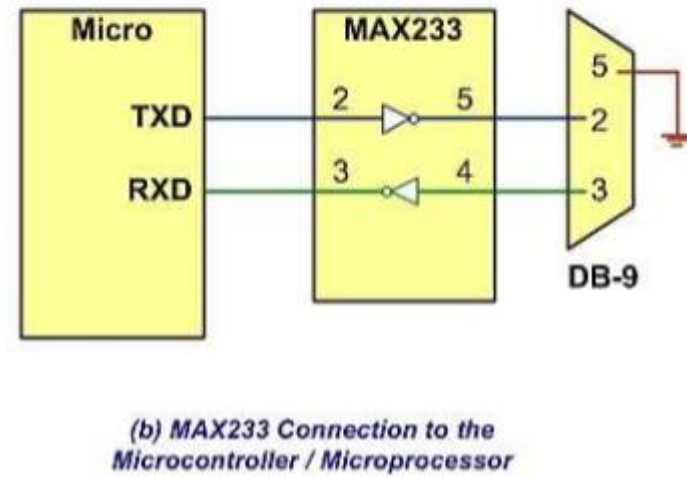
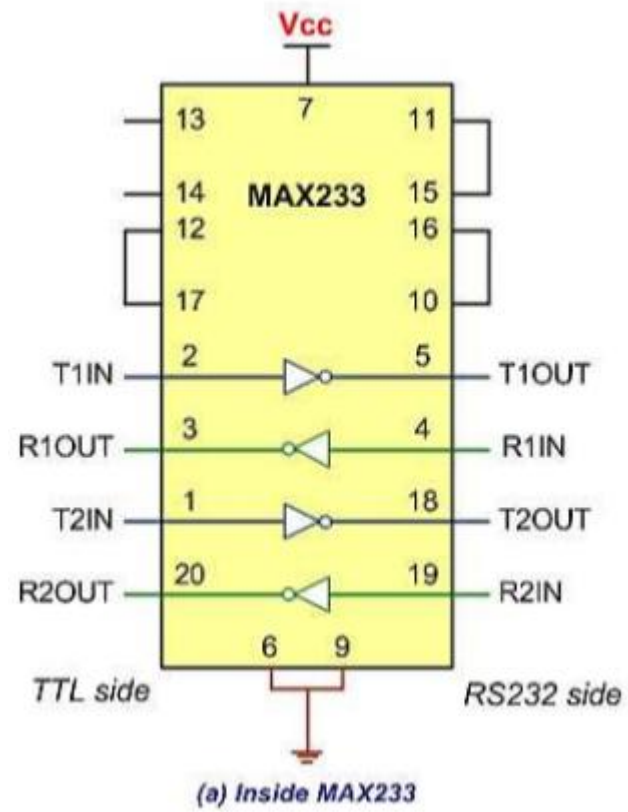
Calculate the time it takes to download a movie of 2 gigabytes using a telephone line. Assume 8 bits, 1 stop bit, no parity, and 57,600 baud rate.

Solution:

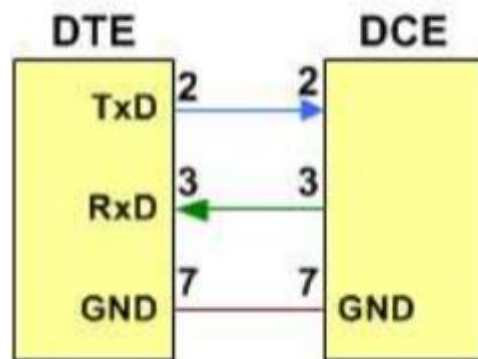
$$2 \times 2^{30} \times 10 / 57,600 = 347,222 \text{ seconds} = \sim 4 \text{ days}$$

Como conectar uma interface serial (sinais com tensões positivas e negativas) a um microcontrolador (sinais lógicos – GND e VCC)?





DTE- DCE Connection



DTE- DTE Connection

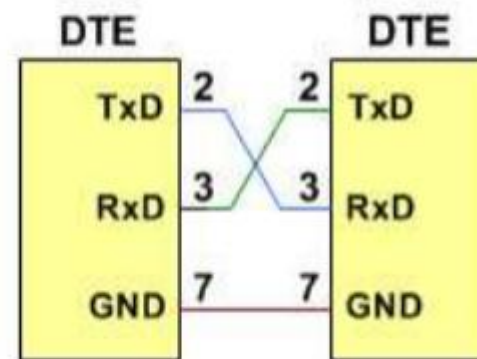


Figure 10-9: DTE-DCE and DTE-DTE Connections

Forma de conectar um computador ao STM32F103C8

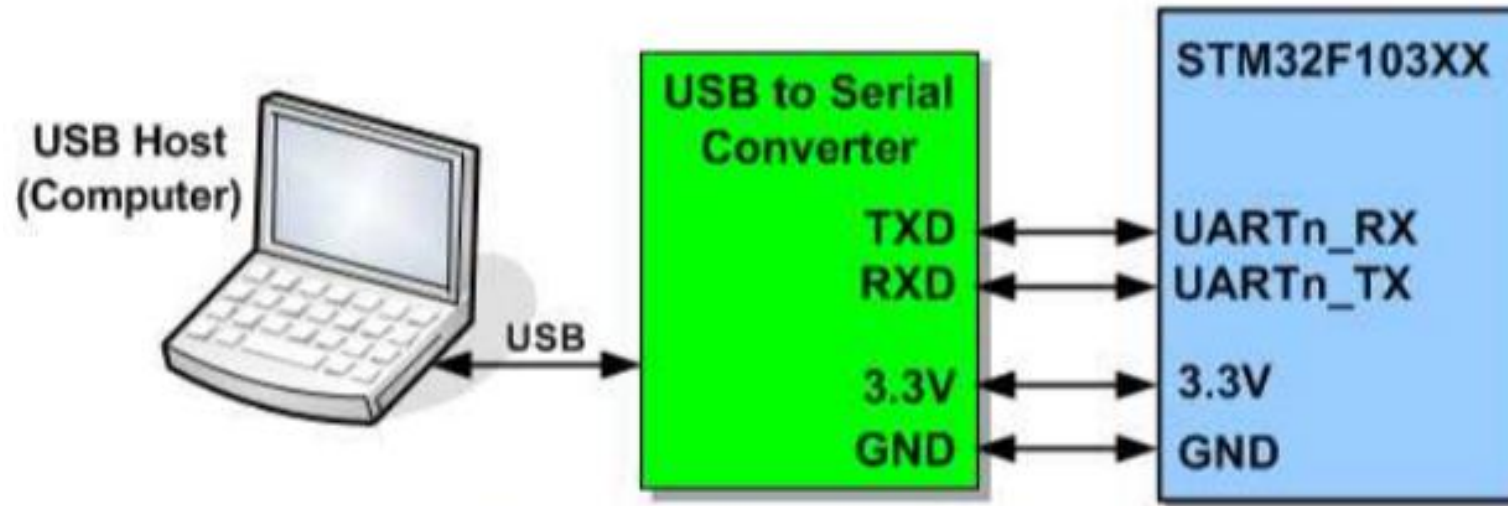


Figure 10-10: Connecting to the PC

USART no STM32F103C8

- O STM32F103C8 implementa a USART1, 2 e 3

	Bus	Address
USART1	APB2	0x40013800 – 0x40013BFF
USART2	APB1	0x40004400 – 0x400047FF
USART3	APB1	0x40004800 – 0x40004BFF
USART4	APB1	0x40004C00 – 0x40004FFF
USART5	APB1	0x40005000 – 0x400053FF

Table 10-2: USART Addresses

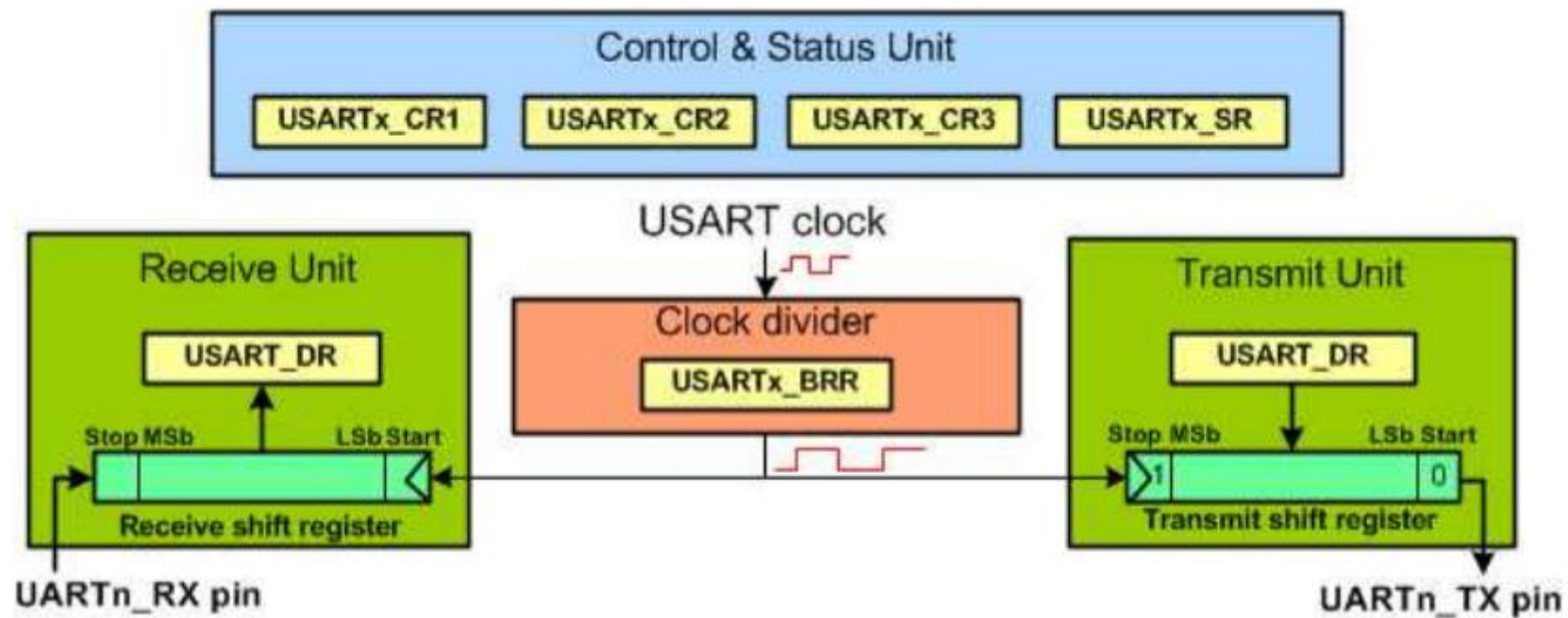


Figure 10-11: a Simplified Block Diagram of UARTn

Registros associados a cada módulo serial

Register name	Offset	Description
USARTx_SR	0x0000	Status register
USARTx_DR	0x0004	Data register
USARTx_BRR	0x0008	Baud rate register
USARTx_CR1	0x000C	Control Register 1
USARTx_CR2	0x0010	Control Register 2
USARTx_CR3	0x0014	Control Register 3

Table 10-3: USART Registers

Definição do Baud Rate

BRR Register and Baud rate

See Figure 10-12. The BRR register is used to set the baud rate.

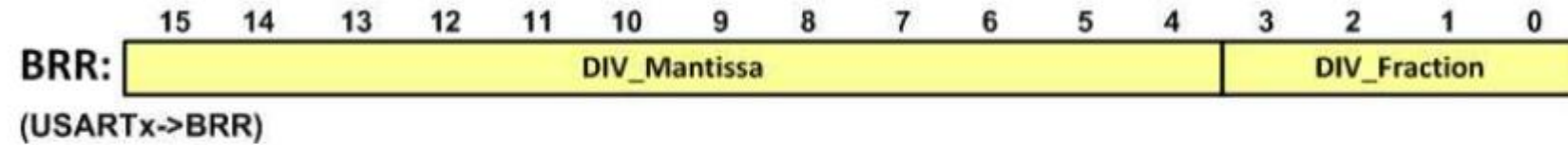


Figure 10-12: USART_BRR

To find the BRR value use the following formula:

$$BRR\ value = \frac{f_{clkx}}{baud\ rate}$$

Example 10-4

In a system, the clock frequency for APB1 and APB2 are 36MHz and 72MHz, respectively. Find the BRR value for:

- (a) USART1 with baud rate of 57600
- (b) USART2 with baud rate of 9600

Solution:

- (a) USART1 is connected to APB2. So, BRR value = 72MHz/57600 = 1250
- (b) USART2 is connected to APB1. So, BRR value = 36MHz/9600 = 3750.

Registros de controle

- São 3 os registros de controle associados a um módulo serial, sendo que os mais importantes entre eles são o Registrador de Controle 1 da USART (USARTx_CR1) e o Registrador de Controle 2 da USART (USARTx_CR2). Eles são registradores de 16 bits. O Registrador de Controle 1 é usado para habilitar a porta serial para enviar e receber dados, entre outras funções.

USART Control 1 (USARTx_CR1) register

The next important register in USART is the control register. We have 3 USART Control Registers. The most important among them are USART Control Register 1 (USARTx_CR1) and USART Control Register 2 (USARTx_CR2). They are 16-bit registers. The Control Register 1 is used to enable the serial port to send and receive data among other things. See Figure 10-13. Notice that the M bit (D12) of the CR1 register determines the framing of data by specifying the number of bits per character. In this textbook, we use the no parity option with a data size of 8 bits. The Control Register 2 can be used to set the number of stop bits. See Figure 10-14. In most applications, 1 stop bit is used (STOP=00). Since the default values of CR2 and CR3 are 0x0000, their default values are proper in most cases.

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
USARTx_CR1:	Reserved	UE	M	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	RWU	SBK	

(USARTx->CR1)

Field	Bit	Description
UE	D13	0 = USART prescaler and outputs disabled. 1 = USART enabled
M	D12	Data format mode bit. We must use this to select 8-bit data frame size 0 = select 8-bit data frame and one start bit 1 = Select 9-bit data frame and one start bit
WAKE	D11	Wake-up condition bit. See the user manual 0 = Idle line wakeup 1 = Address mark wake-up
PCE	D10	Parity Control Enable bit. This will insert a parity bit right after the MSB bit. 0 = no parity bit 1 = parity bit
PS	D9	Parity select (used only if PE is one.) 0 = even parity bit 1 = odd parity bit
PEIE	D8	PE interrupt enable 0 = disabled 1 = A USART interrupt is generated whenever the PE flag of USART_SR is set.
TXEIE	D7	TXE interrupt enable 0 = disabled 1 = A USART interrupt is generated whenever the TXE flag of USART_SR is set.
TCIE	D6	Transmission complete interrupt enable 0 = disabled 1 = A USART interrupt is generated whenever the TC flag of USART_SR is set.

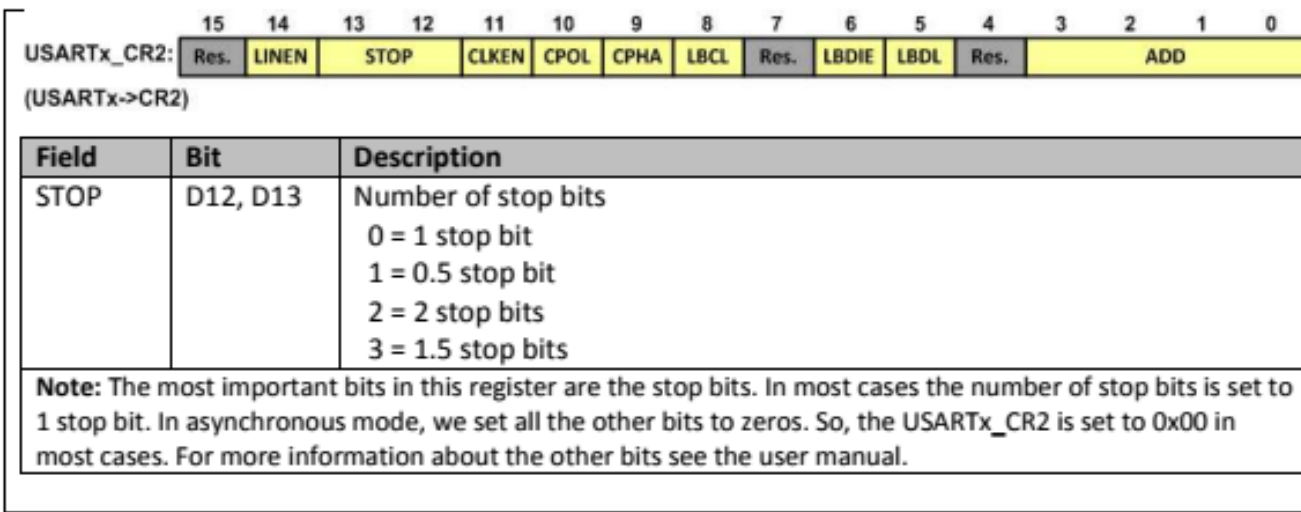


Figure 10-14: USART_CR2

Example 10-5

- (a) What are the values of USART_CR1 and USART_CR2 needed to configure USART2 for 8 data bits (character size), no parity, and 1 stop bit? Enable both receive and transmit.
- (b) Write a program to set the values of USART_CR1 and USART_CR2 for this configuration.

Solution:

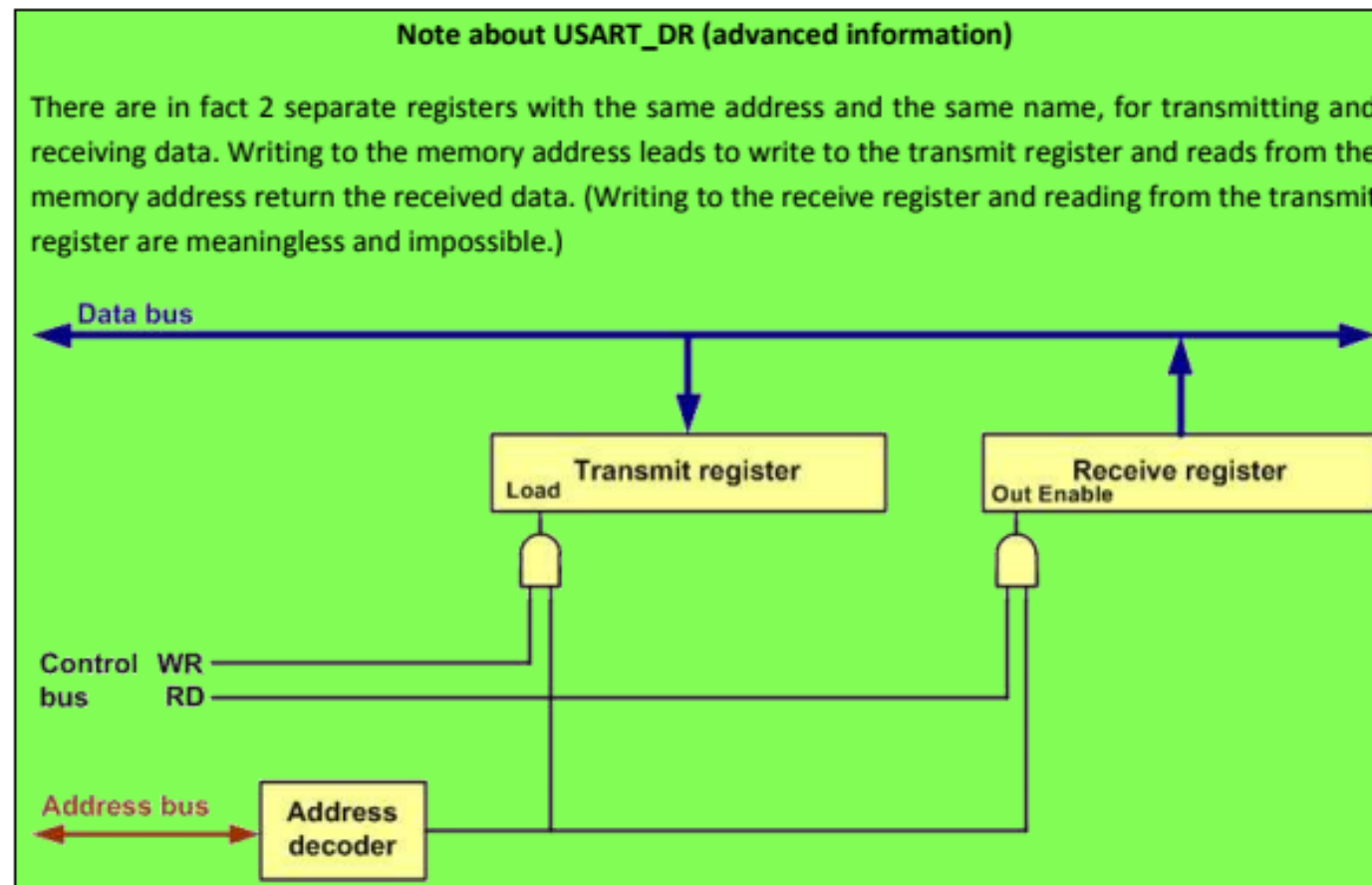
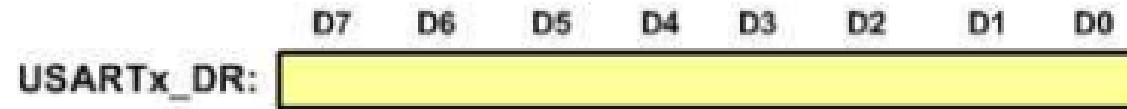
- (a) UE, RE, and TE have to be 1 to enable receive and transmit. M should be 0 for 8-bit data, STOP should be 00 for one stop bit, and PCE should be 0 for no parity.

CR1:	UE	M	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	RWU	SBK
	1	0	0	0	0	0	0	0	0	0	1	1	0	0

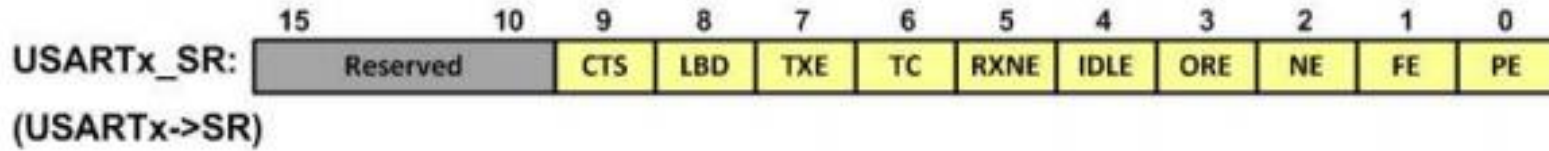
C:

```
USART2->CR1 = 0x200C;
USART2->CR2 = 0;
```

Registro de dados



Registro de Status



Field	Bit	Description
CTS	D9	When the CTS pin is changed the flag is set by hardware. For more information see the user manual.
LBD	D8	LIB break detection flag (See the user manual)
TXE	D7	Transmit data register Empty 0 = Data is not transferred to the shift register yet. 1 = The Data Register is empty and ready for the next byte.
TC	D6	Transmit Complete flag (The flag is set by hardware. To clear the flag, you can write a zero to it. If you read the USART_SR and then write to the USART_DR register, the flag will be automatically cleared.) 0 = Transmission is in progress (shift register is occupied) 1 = Transmission complete (both shift register and Data Register are empty)
RXNE	D5	Receive Data Register not empty flag. This indicates a byte has been received and is sitting in USART Data Register and ready to be picked up. 0 = No data is available in USART Data Register. 1 = Data is available in USART Data Register and ready to be picked up.
IDLE	D4	Idle line detected. See the user manual.
ORE	D3	Overrun error (The flag is set by hardware, if a new byte of data is received while RXNE is set.) 0 = No overrun 1 = Overrun error
NF	D2	Noise error Flag bit (The bit is set by hardware, if noise is detected on the received frame.) 0 = No noise 1 = Noise error
FE	D1	Framing Error bit (The bit is set by hardware, if de-synchronization, noise or a break character is detected.) 0 = No framing error 1 = Framing error
PF	D0	Parity flag error bit (The bit is set by hardware, if the received data has a parity error.) 0 = No parity error 1 = Parity error

Table 10-5: UART Status Register (UARTx_SR)

Pinos físicos associados aos módulos seriais

	Bus	GPIO pin
USART1	TX	PA9
	RX	PA10
USART2	TX	PA2
	RX	PA3
USART3	TX	PB10
	RX	PB11

Table 10-6: TX and RX pins in STM32F103

Example 10-6

Configure the GPIO pins for using USART1.

Solution:

USART1_TX is available on PA9. It needs to be configured as alternate function output push/pull. According to Figure 8-3, the CNF9 bits need to be set to 10 and MODE9 bits should be 11 (or any values other than 00). So, CNF9:MODE9 should be set to 1011 (0x9 in hex). USART1_RX needs to be configured as digital input. It is a good practice to enable the pull-up resistor. The internal pull-up makes the line high if the wire between the microcontroller and the other device is cut.

```
GPIOA->ODR |= (1<<10); //pull-up PA10  
GPIOA->CRH = 0x444448B4; // RX1=input with pull-up, TX1=alt. func output
```

Etapas para transmitir dados

Aqui estão as etapas para configurar um USART e transmitir um byte de dados:

1. Forneça o clock para o USART e para o GPIO escrevendo 1 nos bits apropriados dos registradores **APB1ENR** e **APB2ENR**.
2. Configure o valor do registrador de controle para 1 bit de parada, sem paridade, e tamanho de dado de 8 bits (8/n/1). Habilite o transmissor e o receptor escrevendo **0x200C** no registrador **UARTn_CR1**.
3. Configure a taxa de transmissão (baud rate) do USART usando o registrador **BRR**.
4. Selecione a função alternativa para o pino **UARTn_TX** e configure o **UARTn_RX** como entrada.
5. Escreva um byte no registrador de dados do UART para ser transmitido.
6. Monitore o bit **TC** do registrador de status (**UARTn_SR**) e aguarde até que a transmissão esteja completa.

Exemplo usando HAL

1- Criar novo projeto

2. Ativar a USART

- No diagrama de pinos (pinout) (obs.: feito automaticamente):
 - Clique no pino **PA9** → Selecione **USART1_TX**.
 - Clique no pino **PA10** → Selecione **USART1_RX**.
- Automaticamente o **periférico USART1** será habilitado.

3. Configurar parâmetros da USART

- Vá até Configuration > USART1. (obs.: pode pular a etapa 2 e começar por esta etapa)
- Configure os seguintes parâmetros:
 - Baud Rate: 9600
 - Word Length: 8 Bits
 - Parity: None
 - Stop Bits: 1
 - Mode: TX and RX
 - Hardware Flow Control: None

4- Configurar Clock

- Vá em Clock Configuration.
- Verifique se o clock do periférico USART1 está ativo.
- Certifique-se de que o sistema esteja configurado com 72 MHz (pode ajustar HSE + PLL se quiser máxima performance, ou manter padrão HSI 8 MHz dividido).

5- Gerar Código

6- No seu programa

- Use HAL_UART_Transmit() e HAL_UART_Receive() para enviar e receber dados via USART

```
char msg[] = "Hello via USART!\r\n";
```

```
HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY);
```

Observações

- Para poder fazer uso do printf() é preciso declarar a função

```
int __io_putchar(int ch)
{
    HAL_UART_Transmit(&huart1, (uint8_t *)&ch, 1, HAL_MAX_DELAY);
    return ch;
}
```

- E não esquecer de incluir a biblioteca padrão de entrada saída:
#include “stdio.h”

Observações

- Foi observado um erro durante o processo de depuração com o uso da USART. A possível causa é o não congelamento do SysTick durante o breakpoint. Para que o SysTick também seja congelado em um breakpoint, acrescentar o código abaixo antes da chamada de inicialização de E/S

```
#ifdef DEBUG
    // Se estiver compilando para Debug
    CoreDebug->DEMCR |= CoreDebug_DEMCR_MON_EN_Msk | CoreDebug_DEMCR_MON_PEND_Msk;
    DBGMCU->CR |= DBGMCU_CR_DBG_SLEEP | DBGMCU_CR_DBG_STOP | DBGMCU_CR_DBG_STANDBY;
#endif
```

Quando você usa:

```
printf("alo mundo!\r\n");
```

ou pelo menos `\n`, muitas implementações de `stdio` tratam `stdout` como **line buffered**, ou seja, descarregam o buffer ao encontrar fim de linha. Por isso parece “funcionar imediatamente”.

Soluções:

```
printf("alo mundo!");  
fflush(stdout);
```

ou, melhor para embarcados, desabilitar o buffer logo após inicializar a UART:

```
MX_USART1_UART_Init();  
  
setvbuf(stdout, NULL, _IONBF, 0);
```